

ConstraintLayout

Contents

- **Introduction to ConstraintLayout in Android**
- **Constraint attributes & How to create constraint**
- **Using chains for complex layouts**
- **Using bias for flexible UI design**
- **Using guideline, Flow, Barrier, Chain, etc.**

Introduction – Advantages of ConstraintLayout

- ConstraintLayout provides a flexible and powerful way to create complex layouts in Android, allowing for better control over UI elements positioning and sizing.
- Responsive design: easier to create responsive designs that adapt to different screen sizes and orientations enhancing the user experience across various devices.
- Using ConstraintLayout helps in maintaining a flat view hierarchy, which can lead to improved performance and efficiency in android app development.

Features of ConstraintLayout

- 1. Guidelines for Constraints:** Following proper guidelines for setting constraints ensures consistent and predictable behavior of UI elements within the layout, resulting in a polished user interface.
- 2. Avoiding Nested Layouts:** ConstraintLayout allows for the reduction of nested layouts, which can lead to more efficient UI rendering and improved app performance.
- 3. Utilizing Chains for Alignment:** Chains in ConstraintLayout provide a powerful way to align and distribute UI elements, contributing to a visually appealing and organized layout design.
- 4. Barrier and Group Usage:** Incorporating barriers and groups in ConstraintLayout facilitates the creation of complex and responsive layouts, allowing for efficient management of UI components.

Constraint attributes and creating constraints

- In Android development, a Constraint in ConstraintLayout is a rule or connection that defines how a view should be positioned and sized relative to other views or the parent layout. These constraints allow for flexible and complex UI designs without nested view hierarchies.
- **Anchors:** Each view has anchors on its sides (left, right, top, bottom) and also baseline, center, and more. Constraints are created by connecting these anchors to each other or to the parent container.

Constraint attributes and creating constraints

- **Types of relative Constraints:**
 - Parent Constraint: Connects the view to the edge of the parent layout.
 - Sibling Constraint: Connects the view to another view within the same ConstraintLayout.
- **Type of direction constraint:**
 - Horizontal Constraints: Define the horizontal position of a view.
 - Vertical Constraints: Define the vertical position of a view.
 - **One view have to have at least one horizontal constraint and one vertical constraint.**

Constraint attributes and creating constraints

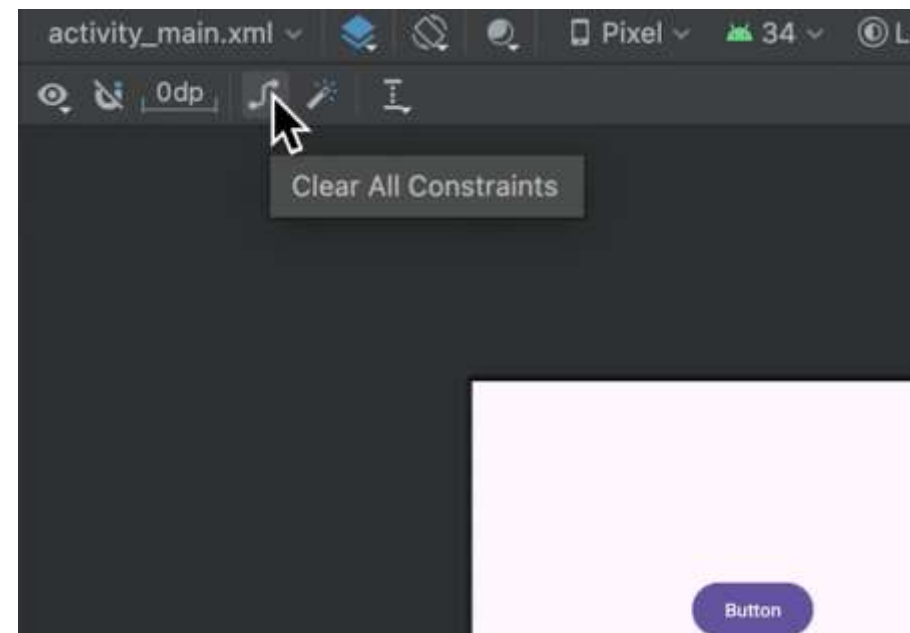
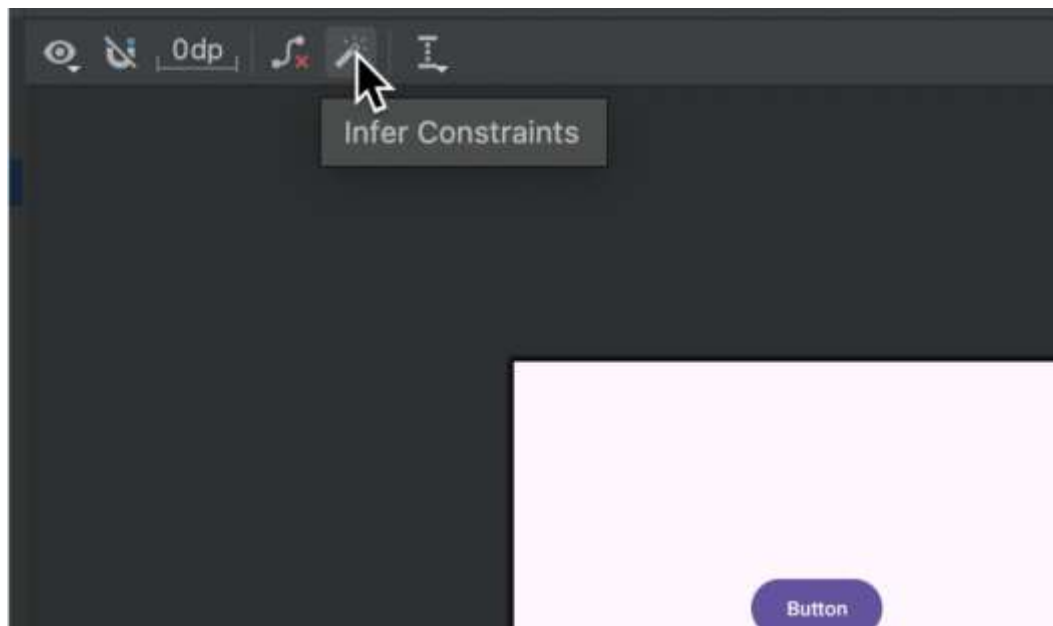
Utilities

- **Bias:** Defines the bias for a view along an axis, which determines how much space is taken up by each side of the constraint. For example, a horizontal bias of 0.5 positions the view in the center horizontally.
- **Chains:** Allows you to create a chain of views that share a common axis, allowing for complex alignment and distribution.
- **Guidelines:** Invisible lines that views can be constrained to, allowing for easier and more flexible design layouts.
- **Flow**
- **Barrier**

Constraint attributes and creating constraints

Create constraints

Method 1: Using Infer constraints: auto-create all constraints

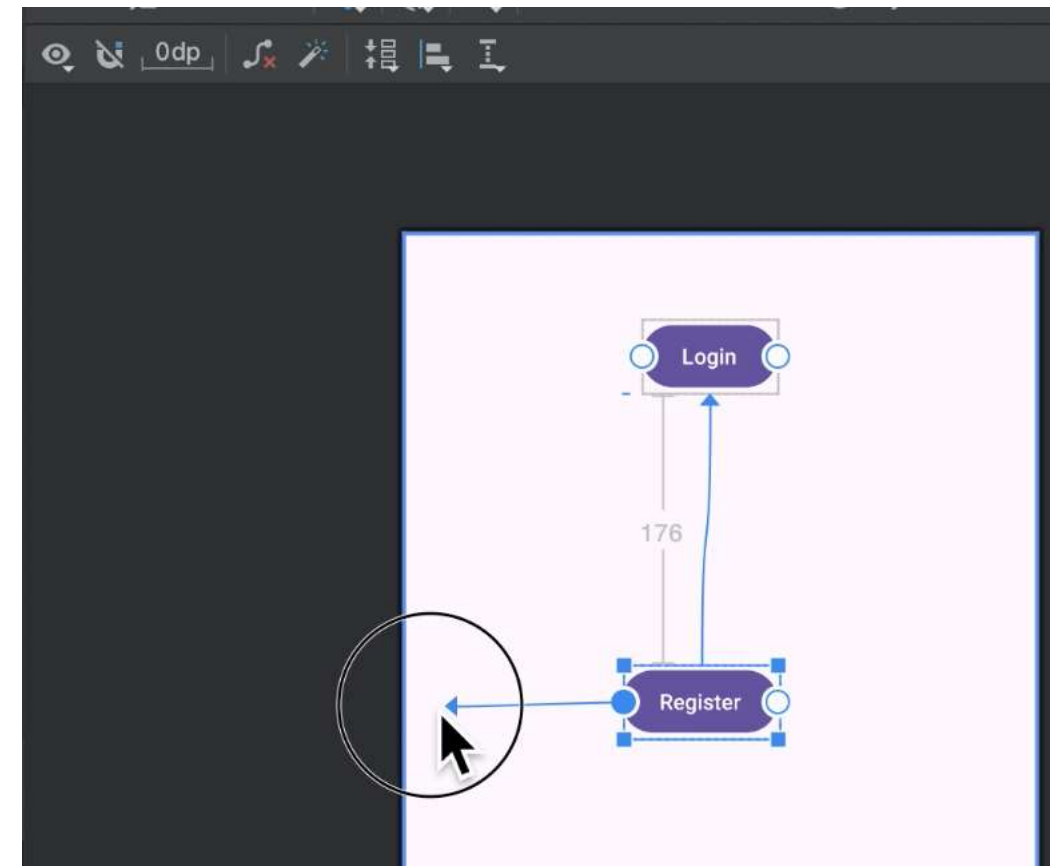


Note: Don't overuse this method. It doesn't responsive

Constraint attributes and creating constraints

Create constraints

Method 2: Using drag and drop 4 anchors

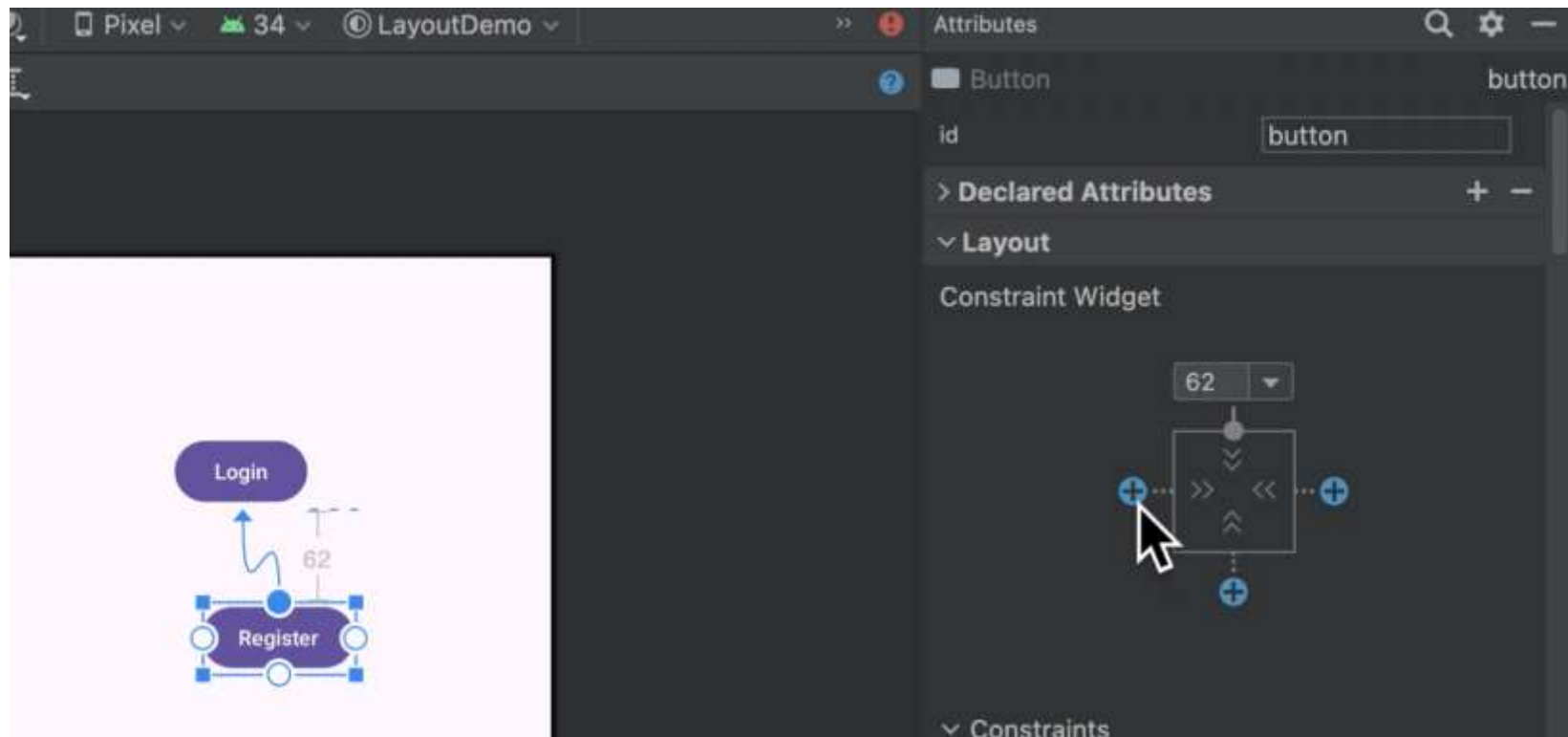


Constraint attributes and creating constraints

Create constraints

Method 3: use constraint simulation table

Example: Create top constraint with nearest view, margin 62dp

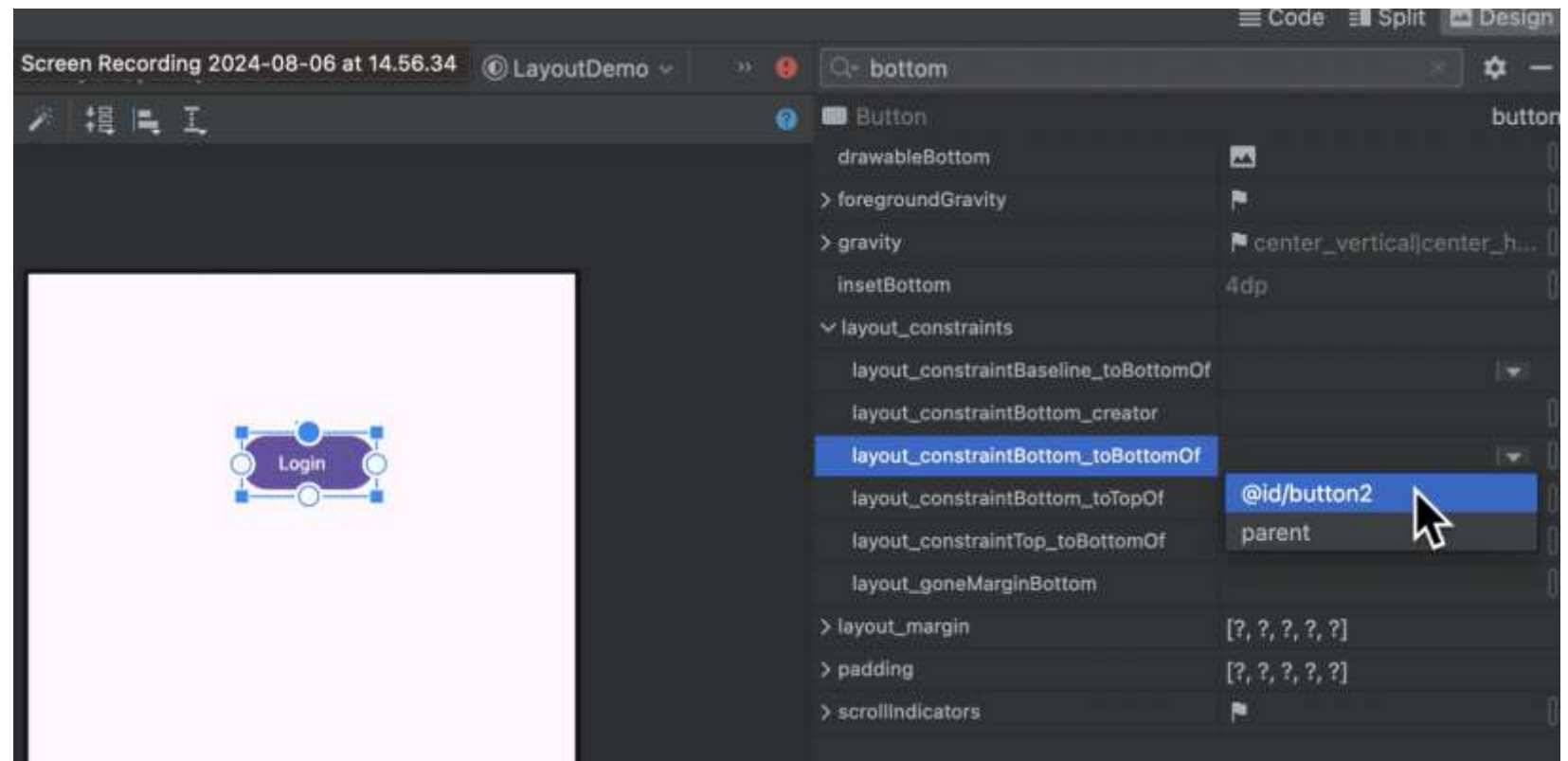


Constraint attributes and creating constraints

Create constraints

Method 4: use constraint attributes from attribute window

Example: assign bottom to bottom of button2 view



Constraint attributes and creating constraints

Create constraints

Method 5: use constraint attributes on XML code

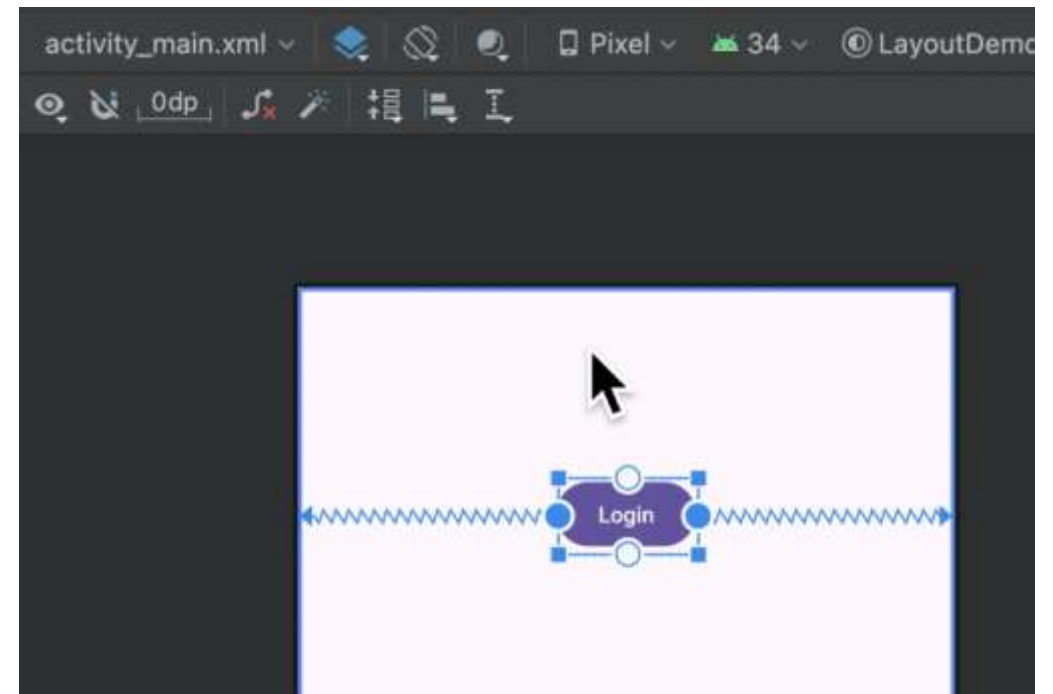
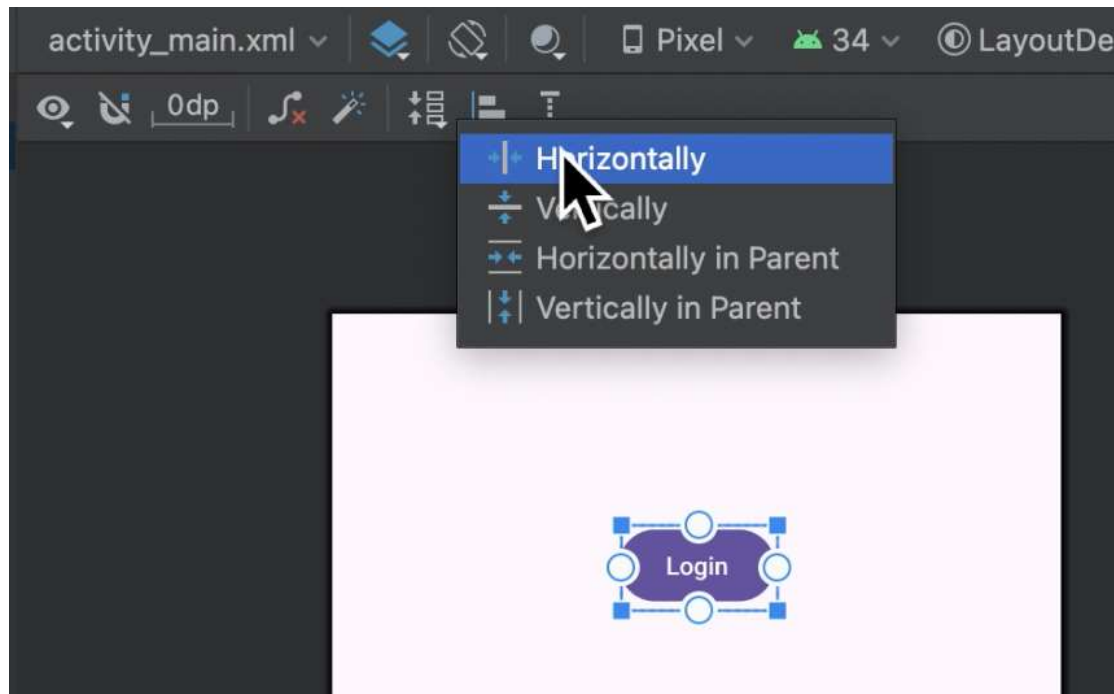
```
<Button  
    android:id="@+id/button"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    android:text="Register"  
  
    app:layout_constraintStart_toStartOf="parent"  
    top/>
```

```
android:layout_marginTop  
android:drawableTop  
android:handwritingBoundsOffsetTop  
android:paddingTop  
<Button  
    app:layout_goneMarginTop  
    app:layout_constraintTop_toTopOf  
    android:firstNameToTopHeight
```

Constraint attributes and creating constraints

Alignment

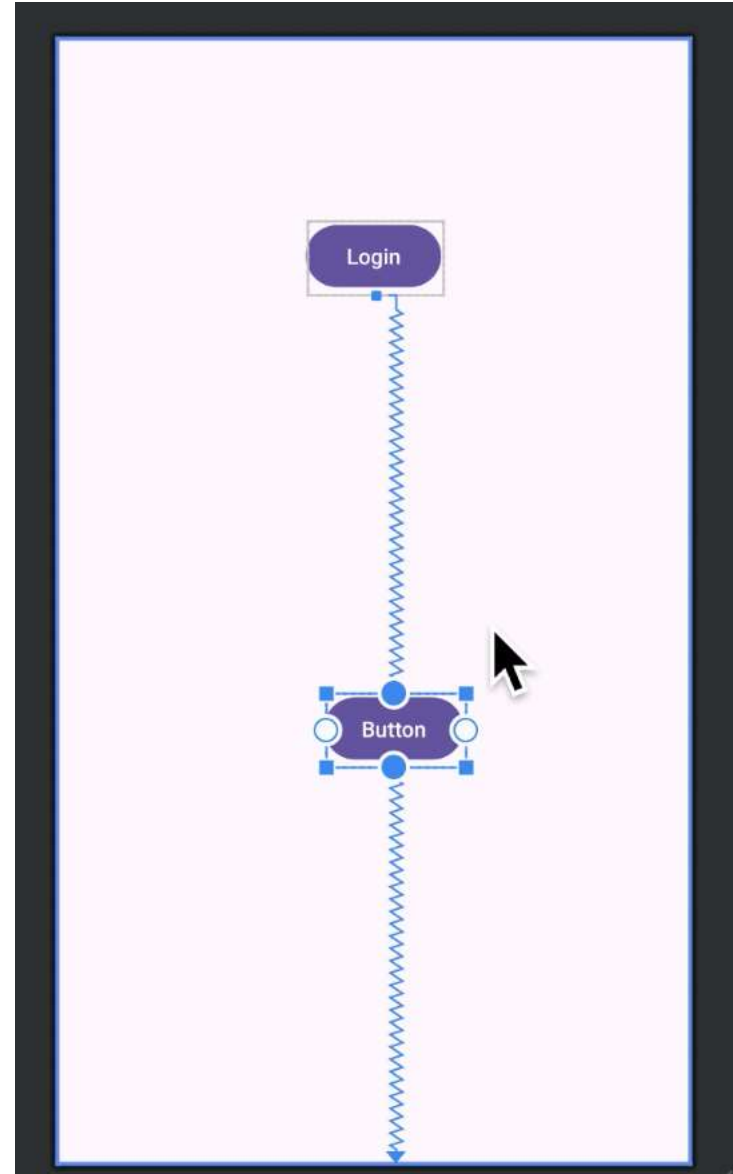
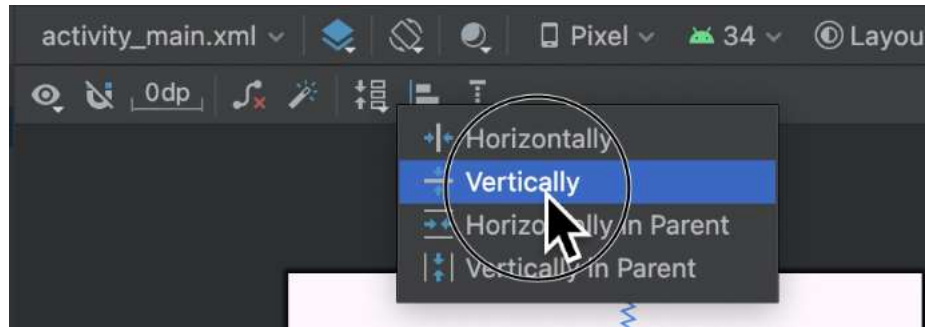
Center in horizontal inside **its constraint**



Constraint attributes and creating constraints

Alignment

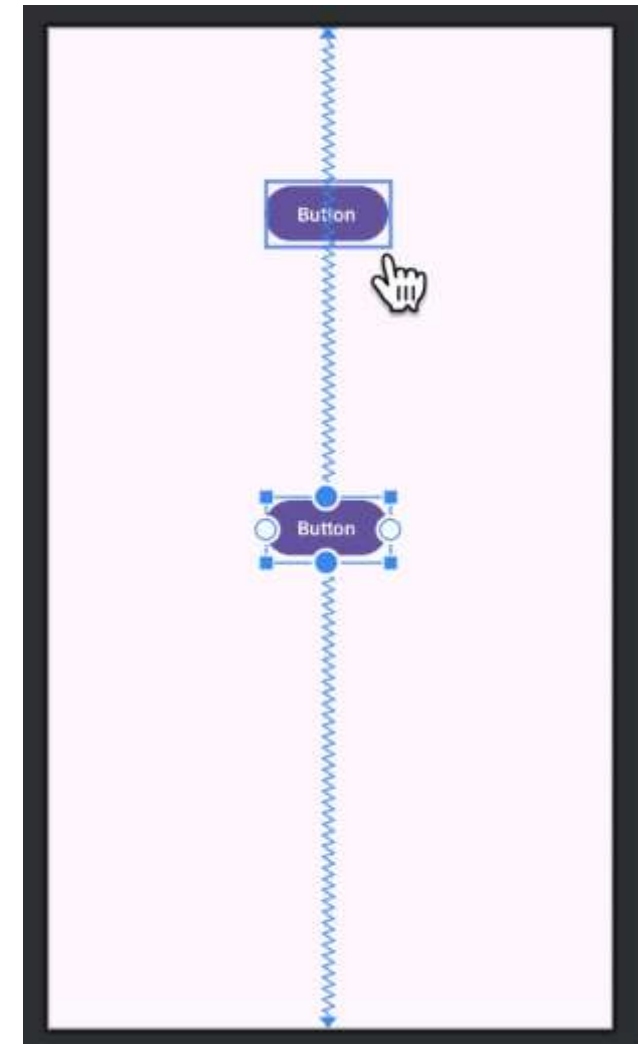
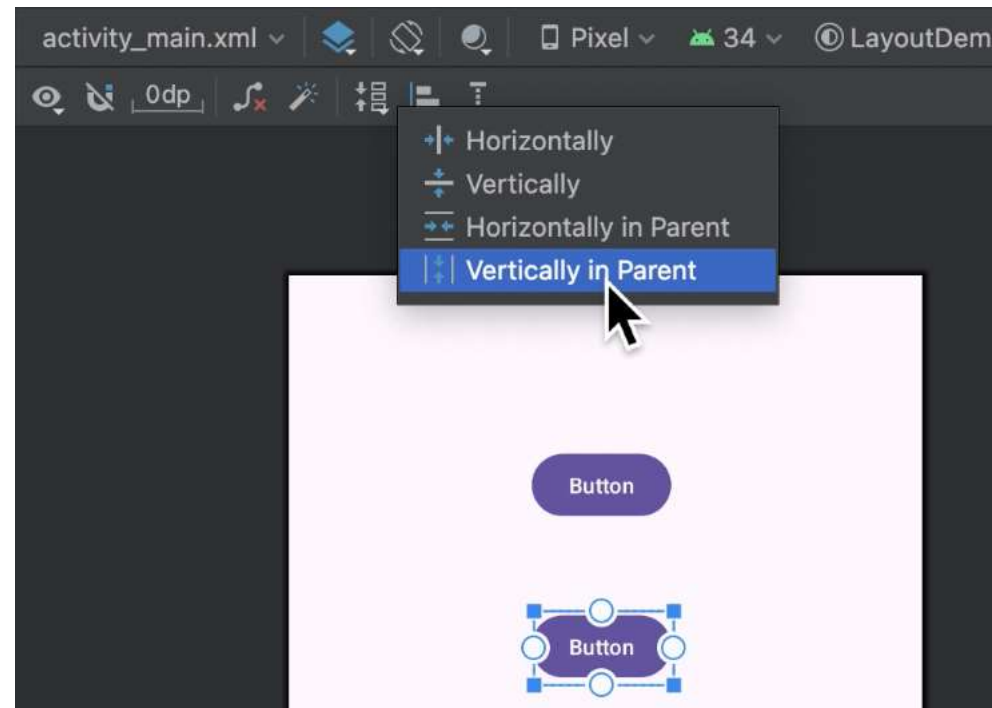
Center in vertical inside **its constraint**



Constraint attributes and creating constraints

Alignment

Center in vertical inside **its parent**



Constraint attributes and creating constraints

Other Alignments

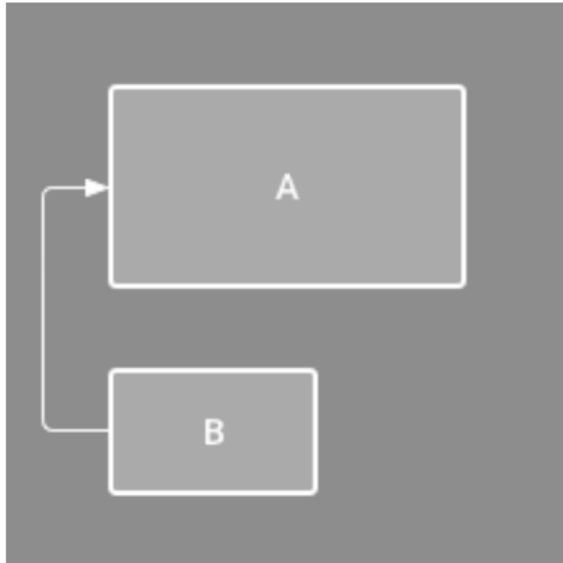


Figure 9. A horizontal alignment constraint.

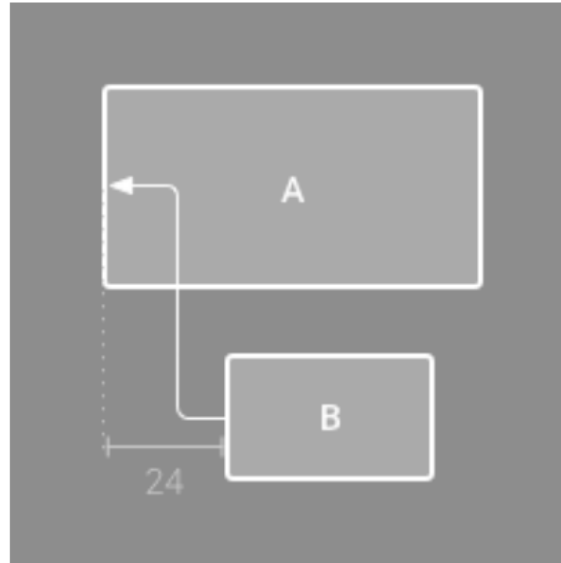


Figure 10. An offset horizontal alignment constraint.

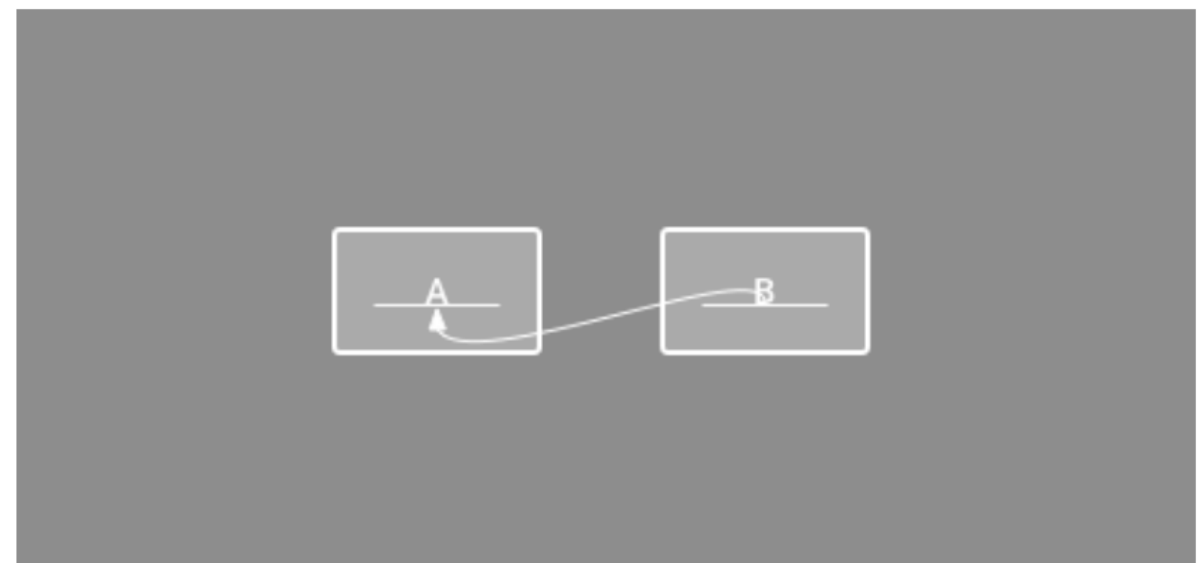
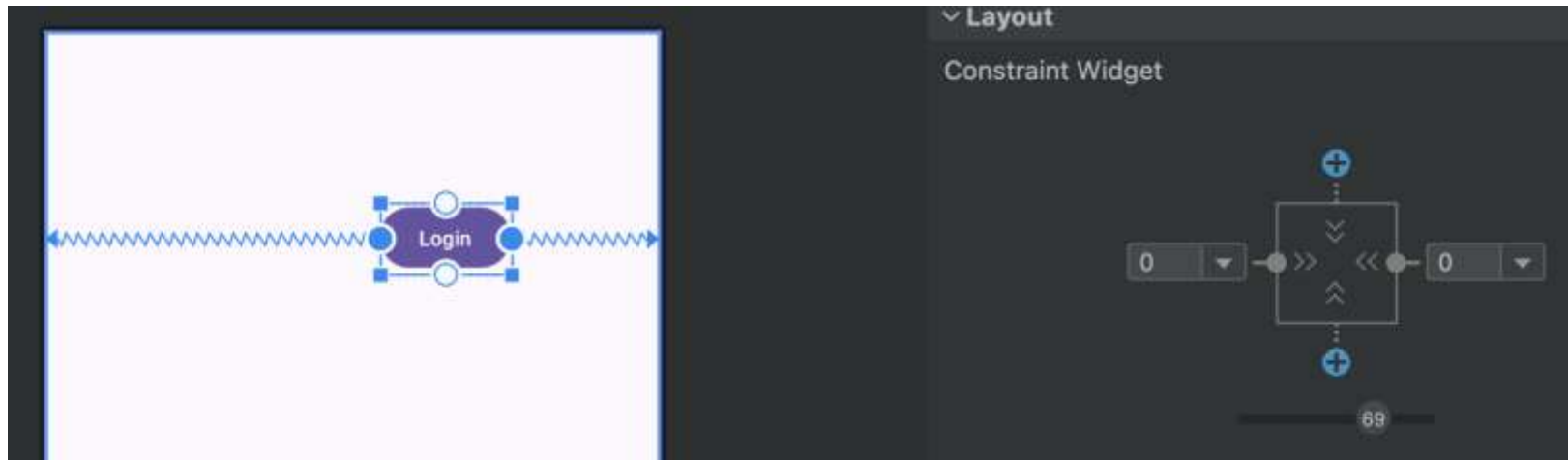


Figure 11. A baseline alignment constraint.

Constraint attributes and creating constraints

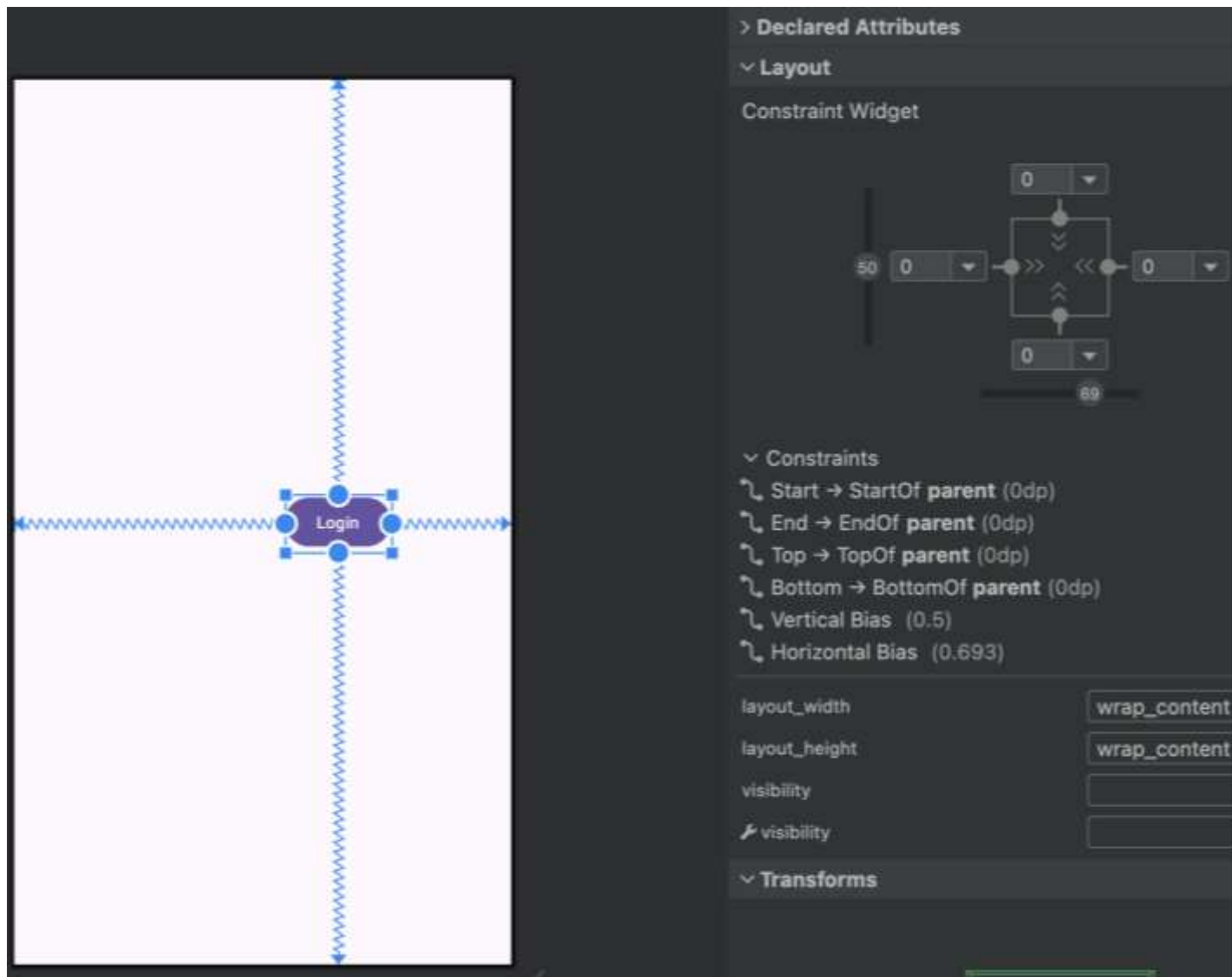
Bias



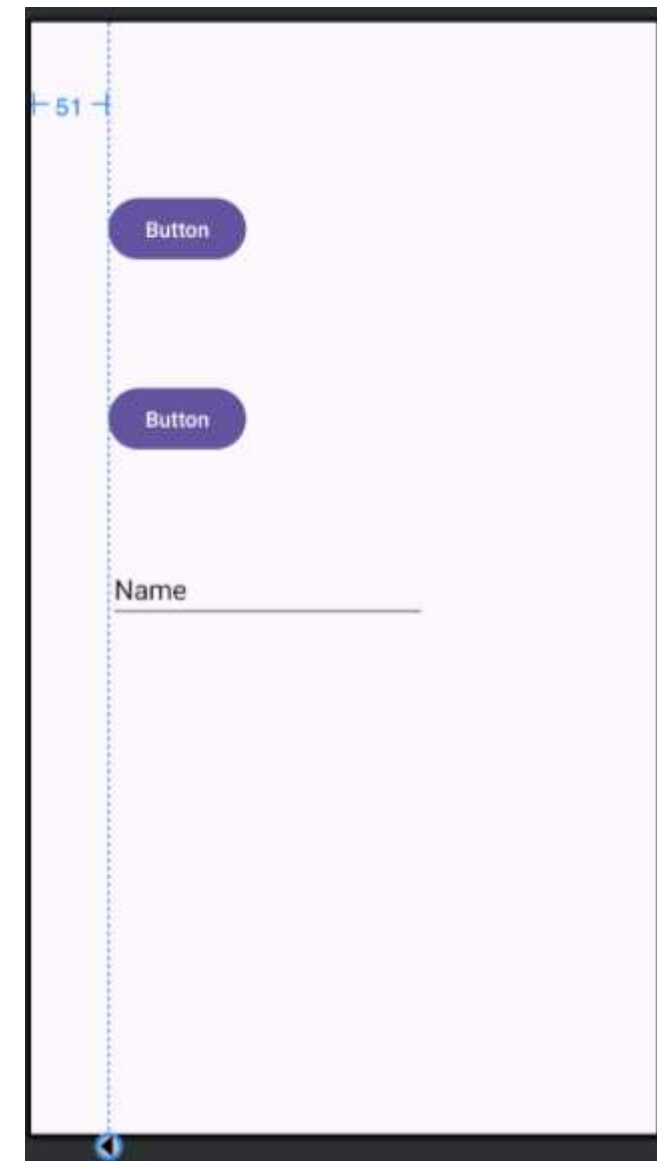
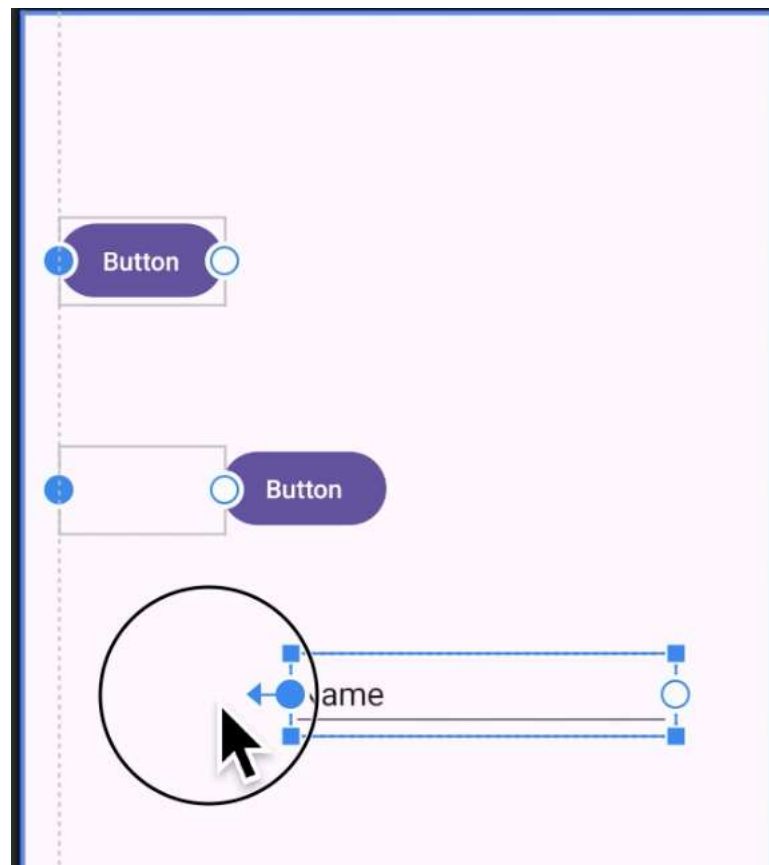
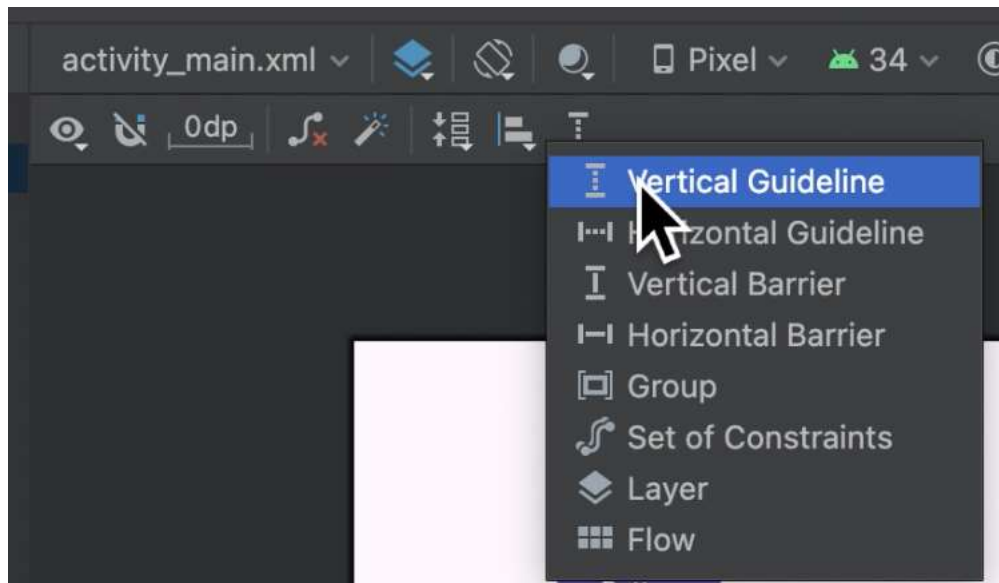
Constraint attributes and creating constraints

Bias:

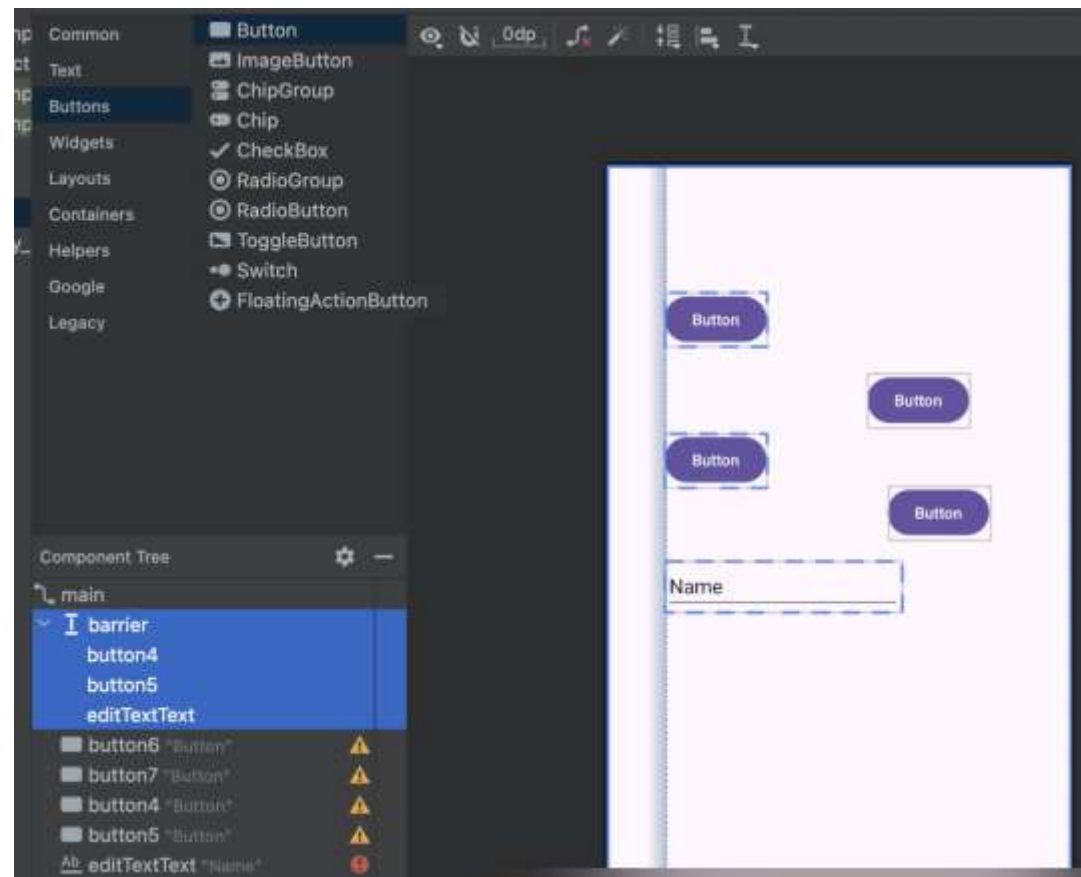
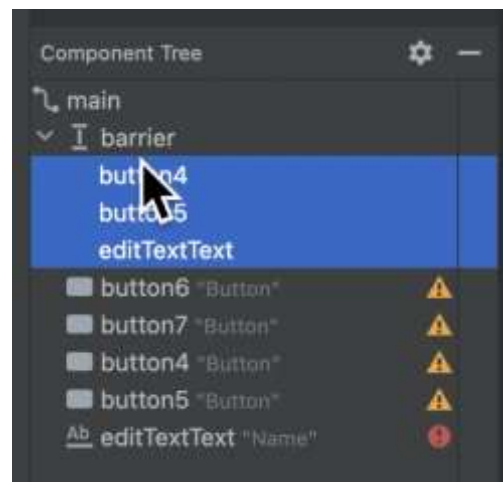
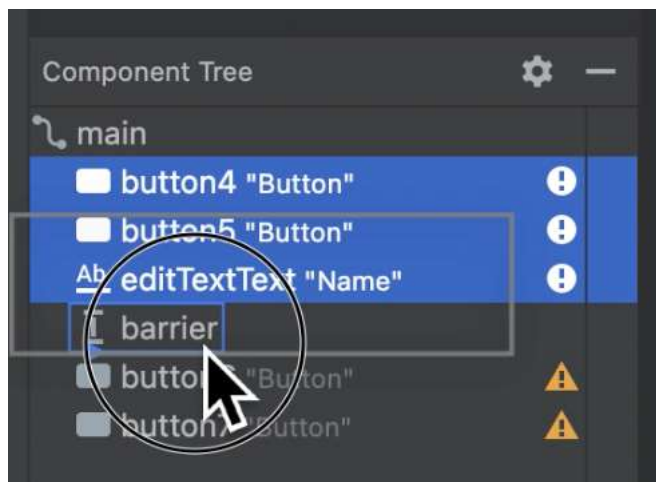
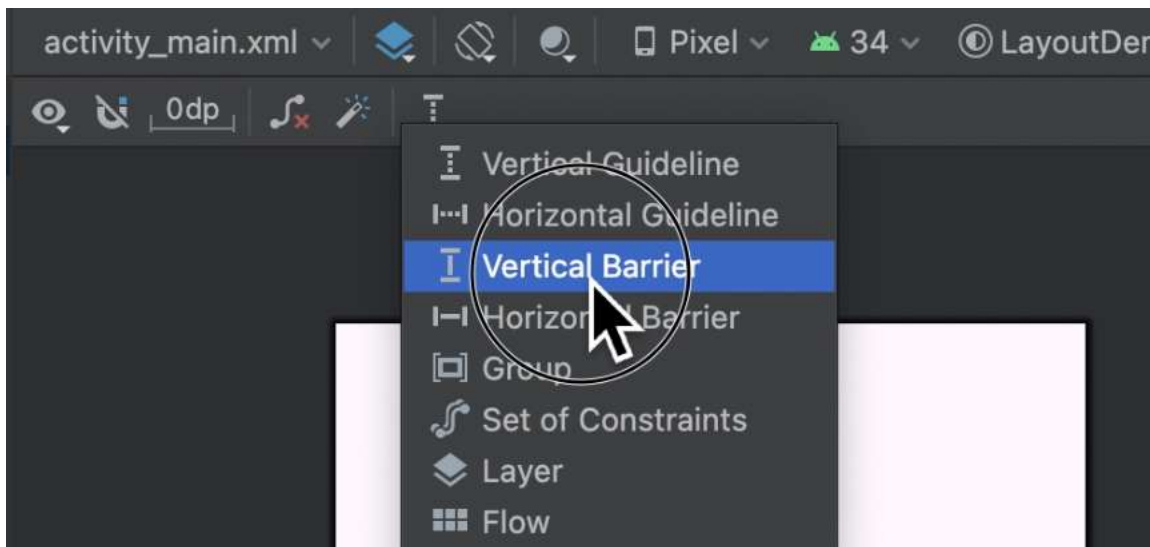
Example: $\text{bias} = 69$,
Mean the position is
at 69% of screen width



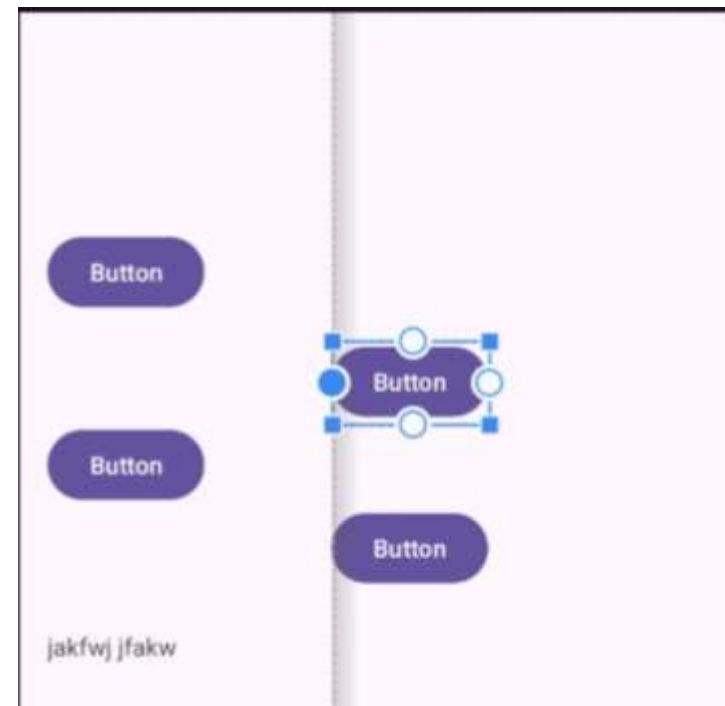
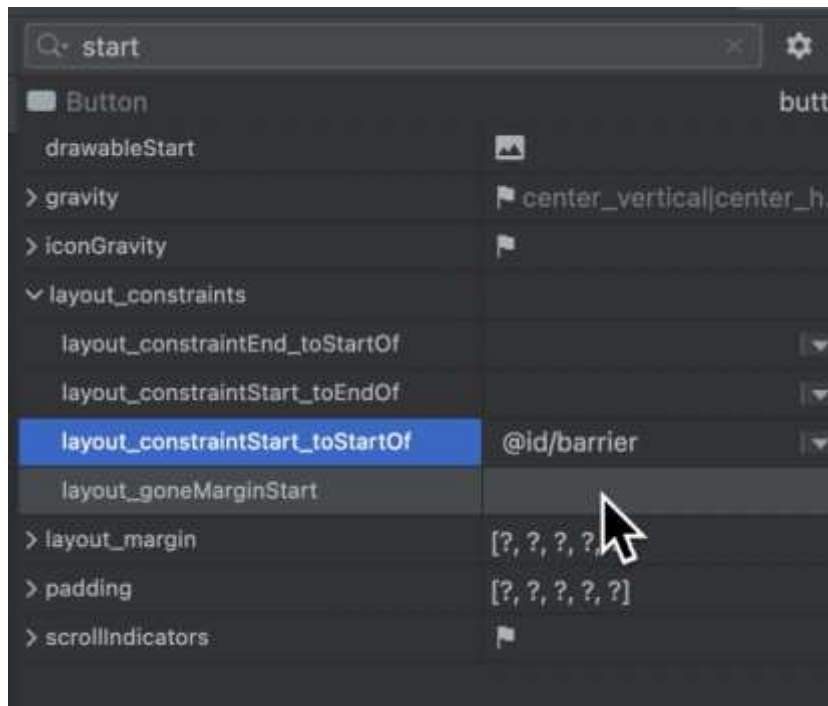
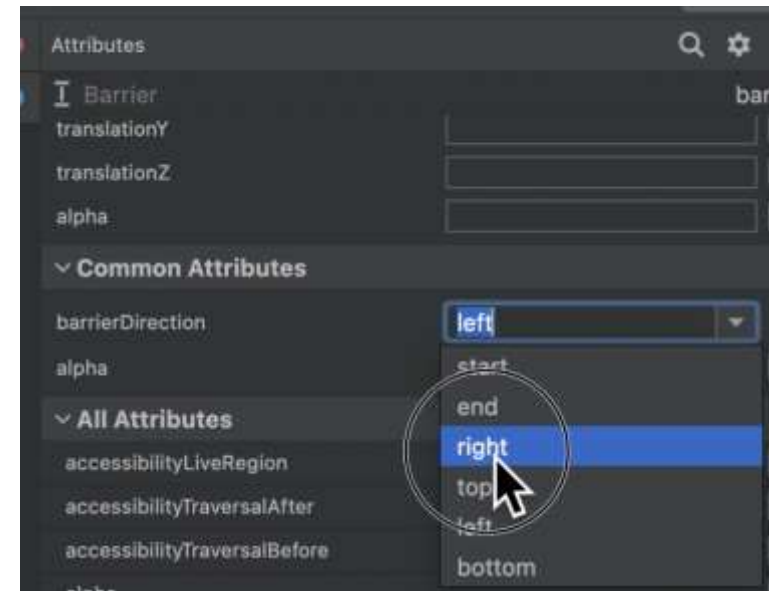
Margin - Using guideline



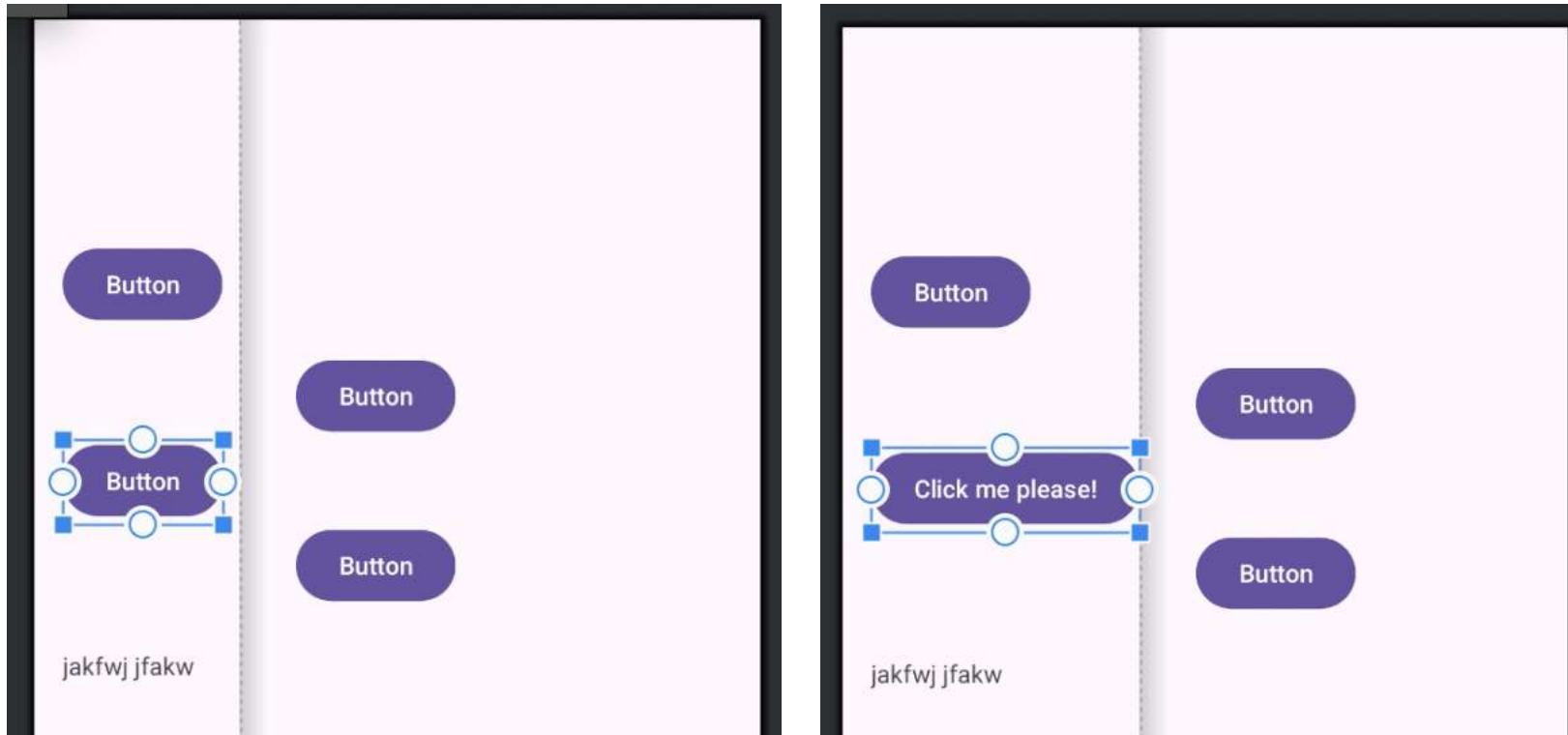
Using barrier



Using barrier



Using barrier

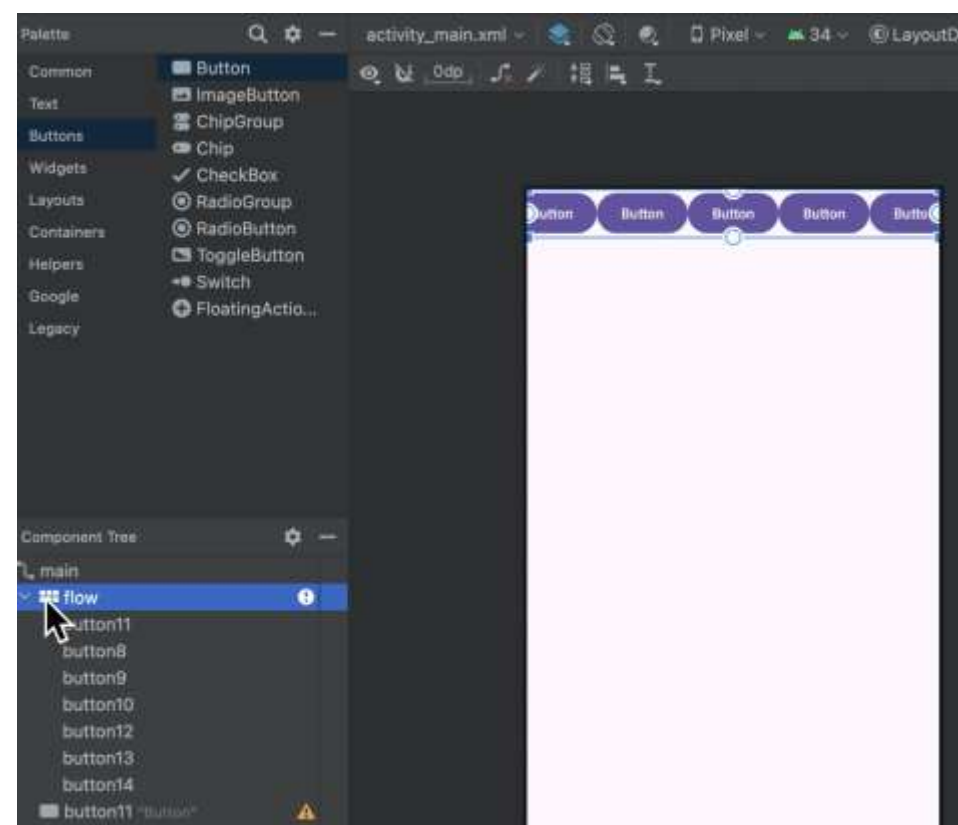
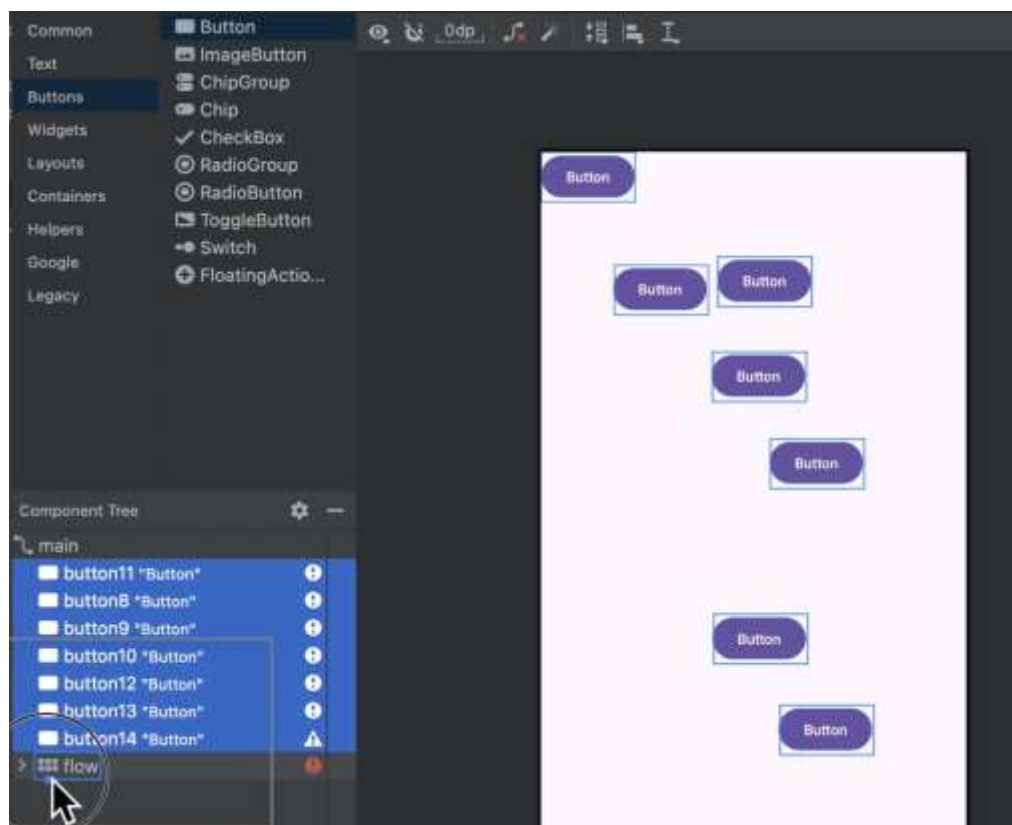
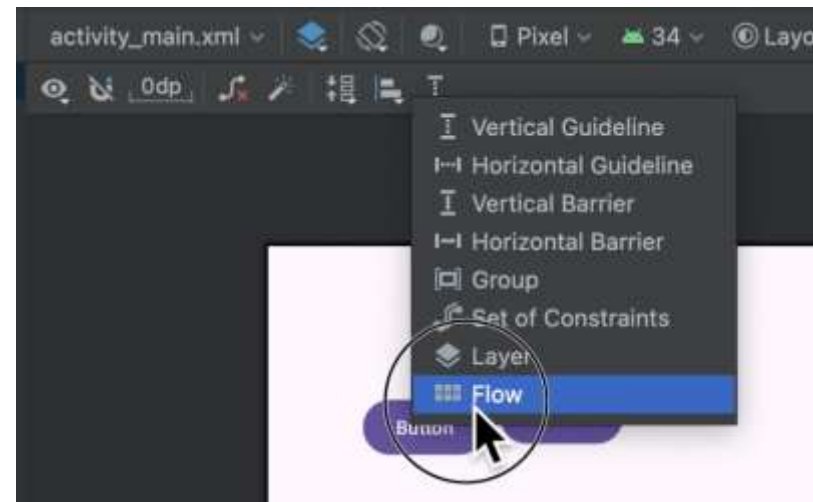


The text of focused button change -> the width change -> the barrier move
➔ the rights buttons move along the barrier.

When the width of any left views change, the left views and the right views never overlap

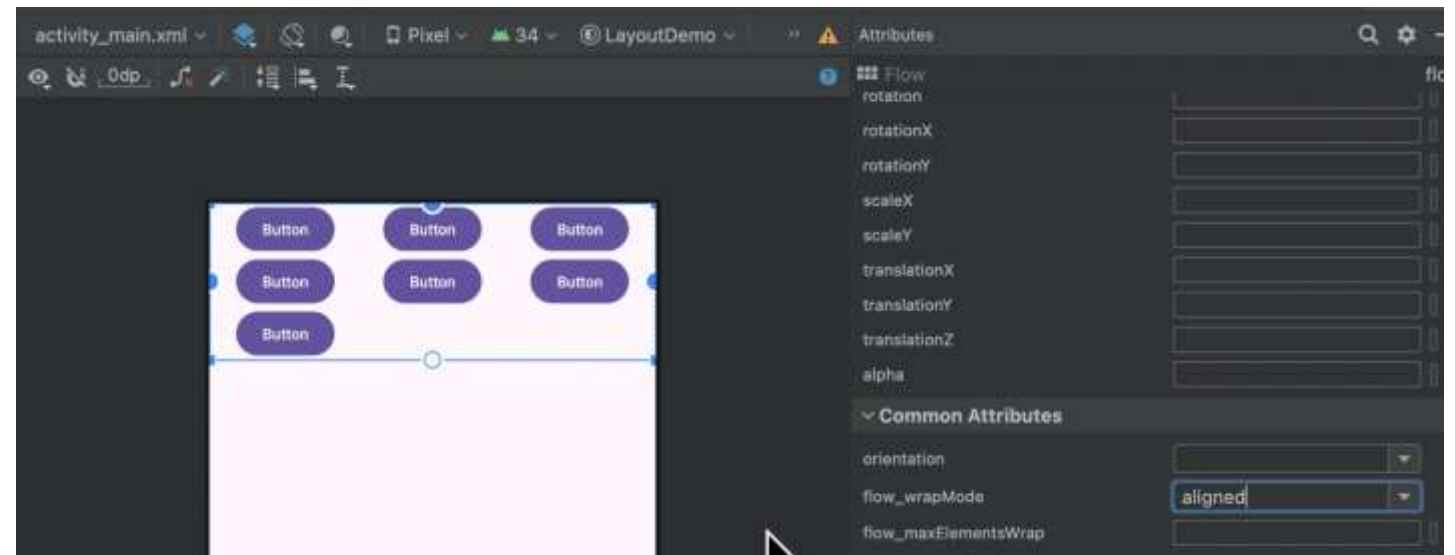
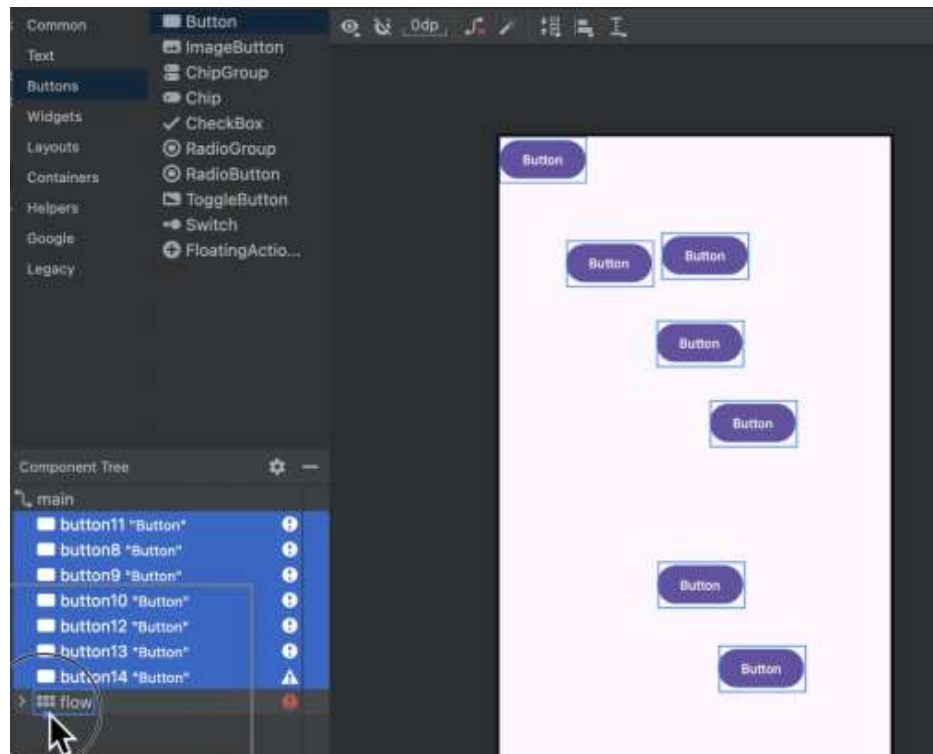
Using Flow

Create flow with wrap-mode: none



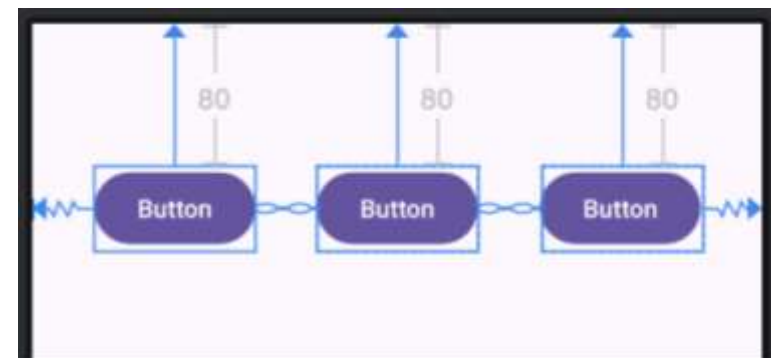
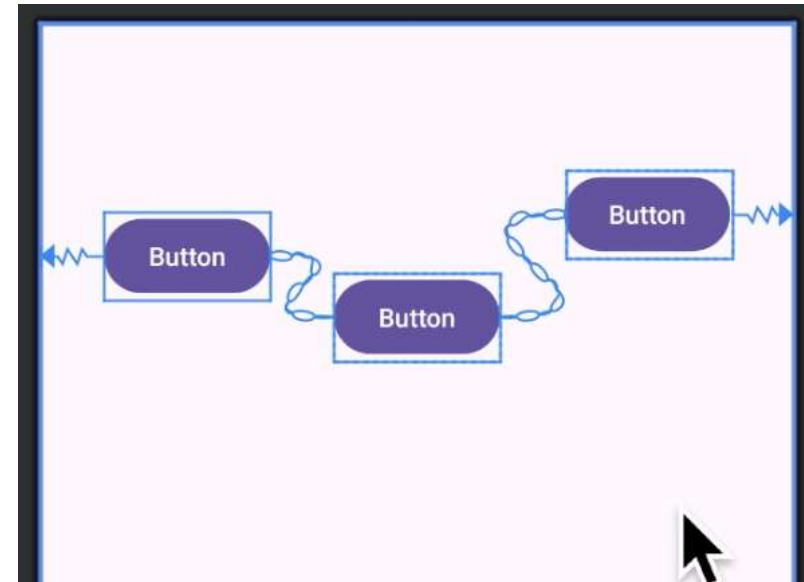
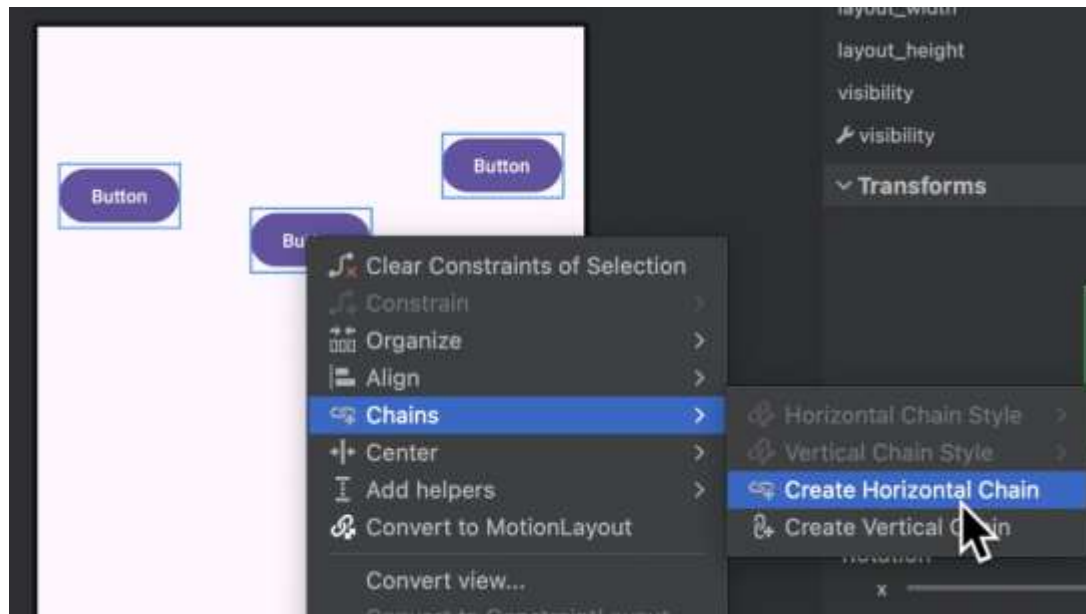
Using Flow

Create flow with wrap-mode: aligned



Using Chain

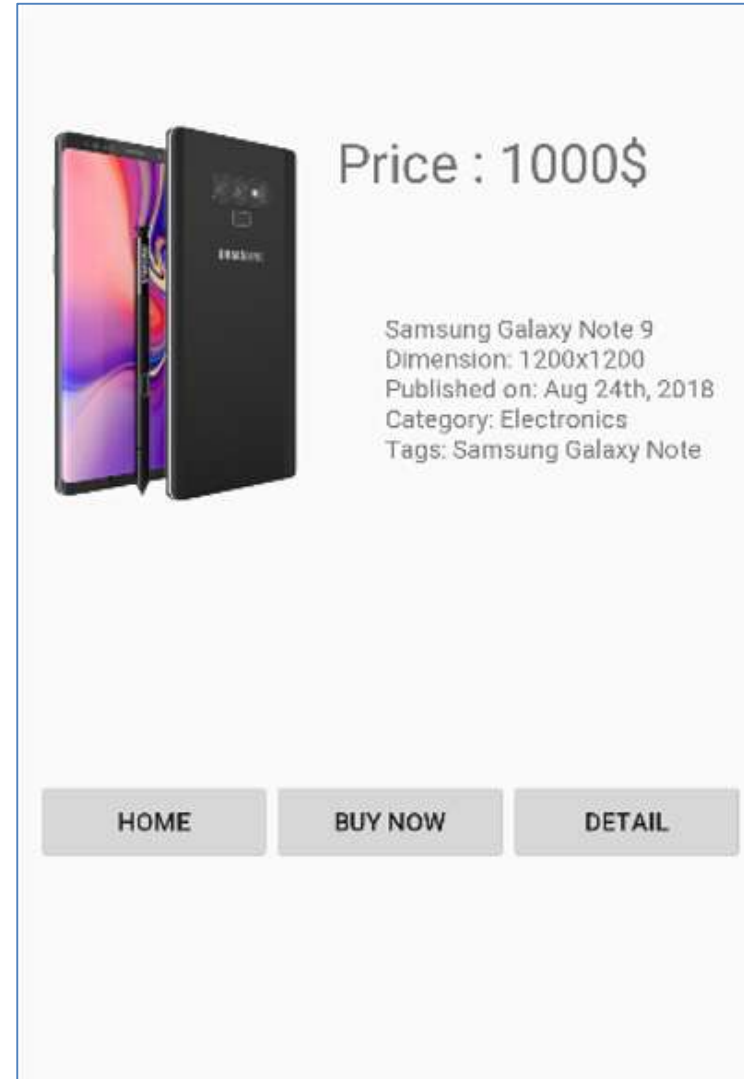
Create flow with wrap-mode: aligned



Others features

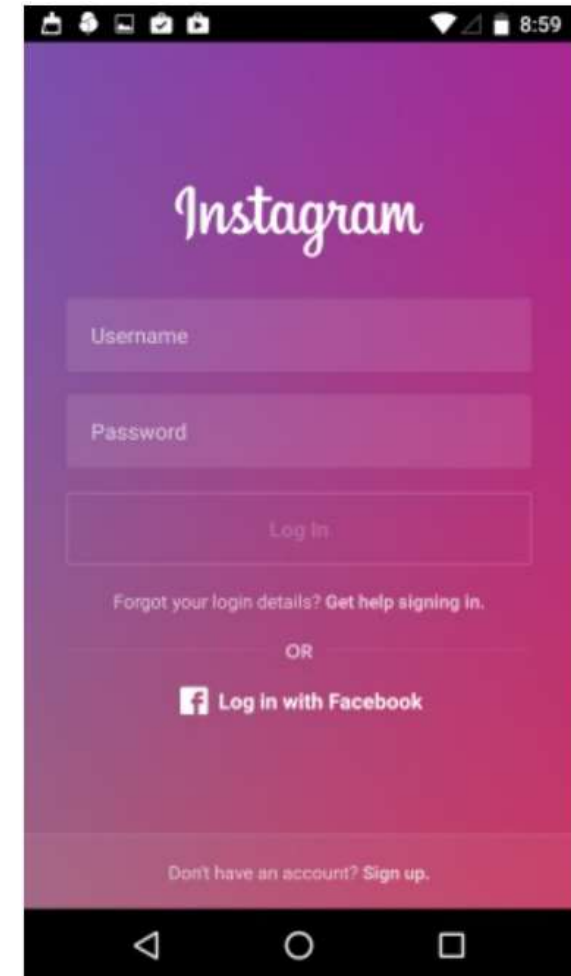
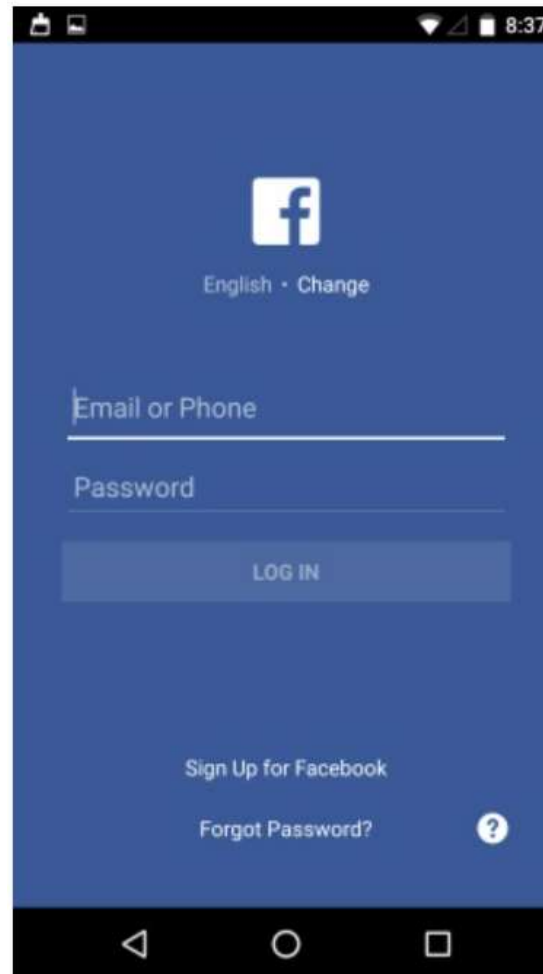
- Function: wrap-content, match-constraint, match-parent, pack
- Group
- Layer

Practice: Let's design!



Practice 2

- Design one of Login Screens as below (no need color & theme)



References

- <https://developer.android.com/develop/ui/views/layout/constraint-layout>